

R&N Moda Feminina

Disciplina: Programação Orientada a Objetos (POO)

Trabalho de G2 – Implementação

Integrantes: Rafaela Linck e Natalia Maciel

Introdução

Este projeto consiste na implementação de um sistema de e-commerce e retaguarda para a loja virtual que criamos (R&N moda feminina). O sistema foi desenvolvido em python, tomando como base os conceitos de orientação a objetos, com uma interface gráfica feita com Tkinter. As telas criadas consistem em: login, navegação de produtos, gerenciamento de carrinho, finalização de compra com geração de nota fiscal e um painel de controle administrativo.

Foram desenvolvidas 8 classes centrais de modelo: Produto, Roupas, Calçado, Usuário, Cliente, Administrador, Carrinho e Pedido.

Criação de diagramas de classe e objetos (Anexamos junto no GitHub).

Estrutura e Modelagem do Sistema

Herança: Utilizamos a herança em duas classes, Produtos e Usuários.

Produto é a superclasse que define os atributos básicos: CÓDIGO, NOME, PREÇO e ESTOQUE.

Roupa e Calçado herdam de produto e acrescentam seus próprios atributos.

Roupa: tamanho(é string pois pensamos em P, M, G), cor e tecido.

Calçado: tamanho (mas é int pois tamanho de calçado é em número), material e marca.

Usuário é a superclasse de autenticação, contém: CPF, NOME, EMAIL, SENHA.

Cliente e Administrador herdam de Usuário e acrescentam seus próprios atributos também.

Cliente: endereço, e também dá acesso a ABA carrinho.

Administrador: cargo, e tem acesso ao painel de gerenciamento do sistema.

Polimorfismo

O método `exibirDetalhes()` está definido na classe base **Produto**.

As subclasses Roupas e Calçado sobrescrevem esse método:

Roupa - `exibirDetalhes()` injeta o tamanho(M,G,P), cor e tecido.

Calçado - `exibirDetalhes()` faz o mesmo, inserindo tamanho numérico, material e marca.

Associação e Agregação:

Cliente e carrinho - A classe **CLIENTE** possui um atributo de **CARRINHO** pois o cliente *tem* um carrinho.

Carrinho e produto - A classe **CARRINHO** pode possuir um ou mais **PRODUTOS**, na quantidade solicitada pelo cliente.

Pedido, cliente e produtos - A classe **PEDIDO** possui uma associação direta com o **CLIENTE**(para saber quem comprou e qual o endereço de entrega) e uma lista de referências aos **PRODUTOS** adquiridos.

Descrição para criação do Diagrama:

Produto (Classe Mãe)

Atributos: `codigo (int)`, `nome (str)`, `preco (float)`, `estoque (int)`.

Métodos:

`exibirDetalhes()` -> Retorna os dados cadastrais básicos.

`diminuirEstoque(quantidade: int)` -> Reduz o estoque do produto se houver quantidade disponível.

`aumentarEstoque(quantidade: int)` -> Adiciona itens ao estoque.

Roupa (Subclasse)

Atributos: Herda Produto, mais `tamanho (str)`, `cor (str)`, `tecido (str)`.

Métodos:

`exibirDetalhes()` -> (Sobrescrito/Polimórfico).

Calçado (Subclasse)

Atributos: Herda Produto, mais `tamanho (int)`, `material (str)`, `marca (str)`.

Métodos:

`exibirDetalhes()` -> (Sobrescrito/Polimórfico).

Usuario (Classe Mãe)

Atributos: `cpf (str)`, `nome (str)`, `email (str)`, `senha (str)`.

Métodos:

`fazerLogin(email: str, senha: str)` -> Valida as credenciais.

Cliente (Subclasse)

Atributos: Herda Usuario, mais endereco (str), carrinho (Carrinho).

Métodos:

adicionarAoCarrinho(produto: Produto, qtd: int) -> Adiciona item ao carrinho.

finalizarCompra() -> Pedido: Transforma os itens do carrinho em um pedido de compra, atualiza o estoque e limpa o carrinho.

Administrador (Subclasse)

Atributos: Herda Usuario, mais cargo (str).

Métodos: Herda todos os métodos de Usuario (como fazerLogin). Permite acesso administrativo para cadastrar novos produtos e gerenciar o status de pedidos de clientes.

Carrinho

Atributos: itens (dict), valorTotal (float).

Métodos:

adicionarItem(produto: Produto, qtd: int) -> Valida estoque e insere o produto.

removerItem(idProduto: int) -> Remove o produto do carrinho.

calcularTotal() -> fSoma o valor de todos os itens baseados no preço.

limpar() -> Esvazia o carrinho.

Pedido

Atributos: idPedido (int), cliente (Cliente), produtosPedidos (list), status (str).

Métodos:

alterarStatus(novoStatus: str): Altera a situação da entrega (ex: Pendente -> Enviado -> Entregue).

atualizarEstoque(quantidade: int): Dá baixa no estoque de cada produto associado ao pedido.

gerarNotaFiscal() -> Formata uma nota fiscal eletrônica em string detalhada e imprime no console do Python.